



UNIVERSIDADE FEDERAL DO MARANHÃO - UFMA
CENTRO DE CIÊNCIA EXATAS E TECNOLOGIA - CCET
ENGENHARIA DA COMPUTAÇÃO - EECp
ARQUITETURA DE COMPUTADORES

Sistema Embarcado de Monitoramento Ambiental e Meteorológico

Discentes:

João Henrique Calixto Teixeira (2023030390)

Marcos Paulo Dominice Silva (2023030318)

Rai Abreu Machado (2023034498)

Ronald Loureiro Camara (2023034460)

Wesley Ferreira Costa (2023034442)

Docente:

Luiz Henrique Neves Rodrigues

São Luís - MA
2026

Sumário

1	Cronograma	2
2	Materiais e Metodologia	2
2.1	Materiais	2
2.2	Metodologia	3
3	Resultados e Discussão	11
3.1	Análise da Temperatura	11
3.2	Análise da Umidade Relativa do Ar	12
3.3	Análise da Pressão Atmosférica	13
3.4	Análise da Luminosidade	13
3.5	Análise da Concentração de Gases em PPM	14
3.6	Discussão Geral dos Resultados	15

1 Cronograma

O desenvolvimento do projeto será realizado conforme o cronograma a seguir:

Abril

- 22/04 (Qua) – Estudo teórico da arquitetura do microcontrolador ATmega328P.
- 27/04 (Seg) – Continuação do estudo teórico.
- 29/04 (Qua) – Definição dos protocolos de comunicação do projeto.

Maio

- 04/05 (Seg) – Planejamento do circuito.
- 06/05 (Qua) – Início da montagem em protoboard.
- 11/05 (Seg) – Integração dos sensores.
- 13/05 (Qua) – Continuação da integração dos sensores.
- 18/05 (Seg) – Desenvolvimento do firmware.
- 20/05 (Qua) – Leitura dos sensores e testes iniciais.
- 25/05 (Seg) – Ajustes no sistema.
- 27/05 (Qua) – Validação do funcionamento.

Junho

- 01/06 (Seg) – Coleta de dados experimentais.
- 03/06 (Qua) – Continuação da coleta de dados.
- 08/06 (Seg) – Organização dos dados.
- 10/06 (Qua) – Análise dos resultados.
- 15/06 (Seg) – Ajustes finais no sistema.
- 17/06 (Qua) – Elaboração do relatório técnico.
- 22/06 (Seg) – Apresentação final do projeto.

2 Materiais e Metodologia

2.1 Materiais

1. **Arduino Nano** – Plataforma de desenvolvimento baseada no microcontrolador ATmega328P, responsável pelo processamento dos dados, gerenciamento dos sensores e execução do firmware do sistema embarcado.

2. **Protoboard** – Placa de prototipagem utilizada para montagem do circuito eletrônico sem a necessidade de soldagem, permitindo conexões rápidas entre os componentes do sistema.
3. **Sensor BME280** – Sensor digital utilizado para medição de temperatura, umidade relativa do ar e pressão atmosférica, além da estimativa aproximada de altitude.
4. **Sensor BH1750** – Sensor digital utilizado para medição da intensidade luminosa ambiente em lux.
5. **Sensor MQ-135** – Sensor analógico utilizado para monitoramento relativo da qualidade do ar através da detecção de gases e compostos voláteis presentes no ambiente.
6. **Jumpers** – Fios condutores utilizados para realizar as conexões elétricas entre o Arduino Nano, sensores e protoboard.
7. **Cabo USB** – Utilizado para alimentação do Arduino Nano e comunicação serial com o computador durante a programação e monitoramento dos dados.
8. **Computador** – Equipamento utilizado para desenvolvimento do firmware, gravação do código no microcontrolador e visualização dos dados através do Monitor Serial da IDE Arduino.
9. **IDE Arduino** – Ambiente de desenvolvimento utilizado para escrita, compilação e upload do firmware para o microcontrolador ATmega328P.

2.2 Metodologia

Inicialmente, foi realizada a montagem do circuito eletrônico em protoboard, integrando os sensores BME280, BH1750 e MQ-135 ao sistema embarcado.

Os sensores BME280 e BH1750 foram conectados utilizando o protocolo de comunicação I2C, compartilhando as linhas SDA e SCL do microcontrolador. Já o sensor MQ-135 foi integrado através de sua saída analógica, conectada ao pino A0 para utilização do conversor analógico-digital (ADC).

A Tabela 1 apresenta as conexões utilizadas no sensor BME280.

Tabela 1: Conexões do sensor BME280

BME280	Conexão
VIN	3V3
GND	GND
SDA	A4
SCL	A5

A Tabela 2 apresenta as conexões utilizadas no sensor BH1750.

Tabela 2: Conexões do sensor BH1750

BH1750	Conexão
VCC	3V3
GND	GND
SDA	A4
SCL	A5
ADDR	GND

O sensor MQ-135 foi conectado conforme apresentado na Tabela 3.

Tabela 3: Conexões do sensor MQ-135

MQ-135	Conexão
VCC	5V
GND	GND
AO	A0

Após a montagem física do circuito, foi desenvolvido o firmware responsável pela inicialização dos sensores, aquisição dos dados ambientais, processamento das informações e exibição dos resultados no Monitor Serial.

Para comunicação com os sensores digitais, foram utilizadas as bibliotecas:

- `Wire.h`;
- `Adafruit_Sensor.h`;
- `Adafruit_BME280.h`;
- `BH1750.h`.

O sensor BME280 foi inicializado utilizando o endereço I2C 0x76, enquanto o BH1750 operou utilizando seu endereço padrão 0x23.

Durante a execução do firmware, o sistema realiza leituras periódicas de:

- temperatura;
- umidade relativa do ar;
- pressão atmosférica;
- altitude aproximada;
- luminosidade ambiente;
- qualidade relativa do ar.

A leitura do sensor MQ-135 foi realizada através da função:

```
analogRead(MQ135_PIN);
```

O valor analógico fornecido pelo sensor é convertido pelo ADC de 10 bits do microcontrolador em um valor digital entre 0 e 1023.

Após a conversão analógico-digital, o firmware realiza uma classificação relativa da qualidade do ar conforme intervalos previamente definidos no código.

A Tabela 4 apresenta os níveis utilizados no sistema.

Tabela 4: Classificação relativa da qualidade do ar

Faixa ADC	Classificação
0 – 40	Boa
41 – 50	Moderada
51 – 60	Ruim
Acima de 60	Crítica

O firmware final utilizado no projeto é apresentado a seguir.

```
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BME280.h>
#include <BH1750.h>
#include <math.h>

// =====
// OBJETOS DOS SENSORES
// =====

Adafruit_BME280 bme;
BH1750 lightMeter;

// =====
// CONFIGURAÇÕES
// =====

#define SEALEVELPRESSURE_HPA (1013.25)
#define MQ135_PIN A0

bool bmeStatus = false;

// Constantes do MQ-135
#define RL_MQ135 1.0 // Resistor de carga (1.0 kOhm)
#define CURVA_A 110.47 // Constante 'a' da curva
#define CURVA_B -2.862 // Constante 'b' da curva

float Ro_MQ135 = 10.0; // Variável para guardar o Ro calibrado

// =====
// FUNÇÕES DE CLASSIFICAÇÃO (ORIGINAIS)
// =====
```

```

// ----- TEMPERATURA -----
String classificarTemperatura(float temp) {
    if (temp <= 15) {
        return "FRIO";
    } else if (temp <= 28) {
        return "MODERADO";
    } else if (temp <= 35) {
        return "QUENTE";
    } else {
        return "MUITO QUENTE";
    }
}

// ----- UMIDADE -----
String classificarUmidade(float umidade) {
    if (umidade < 30) {
        return "SECA";
    } else if (umidade <= 60) {
        return "CONFORTAVEL";
    } else if (umidade <= 80) {
        return "UMIDA";
    } else {
        return "MUITO UMIDA";
    }
}

// ----- PRESSÃO -----
String classificarPressao(float pressao) {
    if (pressao < 1000) {
        return "BAIXA";
    } else if (pressao <= 1020) {
        return "NORMAL";
    } else {
        return "ALTA";
    }
}

// ----- LUMINOSIDADE -----
String classificarLuminosidade(float lux) {
    if (lux < 50) {
        return "ESCURO";
    } else if (lux <= 300) {
        return "MODERADO";
    } else if (lux <= 1000) {
        return "CLARO";
    } else {
        return "MUITO CLARO";
    }
}

```

```

}

// =====
// FUNÇÕES DO MQ-135 (NOVAS)
// =====

// ----- QUALIDADE DO AR (ANVISA RE-09/2003) -----
String classificarQualidadeAr(float ppm) {
    if (ppm <= 450) {
        return "EXCELENTE (Ar Externo/Puro)";
    } else if (ppm <= 1000) {
        return "ADEQUADA (Dentro da Norma)";
    } else if (ppm <= 1500) {
        return "RUIM (Acima do limite ANVISA - Ventile o local)";
    } else {
        return "CRITICA (Risco a Saude)";
    }
}

// ----- CALIBRAÇÃO -----
float calibrarMQ135() {
    Serial.print("Calibrando MQ-135 no ar limpo...");
    float ro_soma = 0;

    for (int i = 0; i < 50; i++) {
        float v_out = analogRead(MQ135_PIN) * (5.0 / 1023.0);
        if (v_out > 0) {
            float rs = ((5.0 - v_out) / v_out) * RL_MQ135;
            ro_soma += rs;
        }
        delay(100);
    }

    float ro_final = ro_soma / 50.0;
    Serial.println(" Concluido!");
    Serial.print("Resistencia Base (Ro) calculada: ");
    Serial.print(ro_final);
    Serial.println(" kOhm");
    Serial.println();

    return ro_final;
}

// ----- CÁLCULO DE PPM -----
float calcularPPM(int leituraADC) {
    float v_out = leituraADC * (5.0 / 1023.0);
    if (v_out <= 0) return 0;

```

```

float r_s = ((5.0 - v_out) / v_out) * RL_MQ135;
float razao = r_s / Ro_MQ135;

float ppm = CURVA_A * pow(razao, CURVA_B);
return ppm;
}

// =====
// SETUP
// =====

void setup() {
  Serial.begin(9600);
  Wire.begin();

  Serial.println("=====");
  Serial.println(" SISTEMA DE MONITORAMENTO AMBIENTAL ");
  Serial.println("=====");

  // =====
  // Inicialização BME280
  // =====
  bmeStatus = bme.begin(0x76);

  if (!bmeStatus) {
    Serial.println("Erro ao encontrar o BME280!");
    Serial.println("Verifique conexoes e endereco I2C.");
    Serial.println();
  } else {
    Serial.println("BME280 inicializado com sucesso!");
  }

  // =====
  // Inicialização BH1750
  // =====
  lightMeter.begin();
  Serial.println("BH1750 inicializado com sucesso!");

  // =====
  // Inicialização MQ-135
  // =====
  pinMode(MQ135_PIN, INPUT);
  Serial.println("MQ-135 inicializado com sucesso!");
  Serial.println();

  // Autocalibração do Ro (Necessário ar limpo na hora de ligar)
  Ro_MQ135 = calibrarMQ135();
}

```

```

// =====
// LOOP PRINCIPAL
// =====

void loop() {
  Serial.println("===== DADOS AMBIENTAIS =====");
  Serial.println();

  // =====
  // BME280
  // =====
  if (bmeStatus) {
    float temperatura = bme.readTemperature();
    float umidade = bme.readHumidity();
    float pressao = bme.readPressure() / 100.0F;
    float altitude = bme.readAltitude(SEALEVELPRESSURE_HPA);

    // TEMPERATURA
    Serial.print("Temperatura: ");
    Serial.print(temperatura);
    Serial.print(" C");
    Serial.print("  [");
    Serial.print(classificarTemperatura(temperatura));
    Serial.println("]");

    // UMIDADE
    Serial.print("Umidade: ");
    Serial.print(umidade);
    Serial.print(" %");
    Serial.print("  [");
    Serial.print(classificarUmidade(umidade));
    Serial.println("]");

    // PRESSÃO
    Serial.print("Pressao: ");
    Serial.print(pressao);
    Serial.print(" hPa");
    Serial.print("  [");
    Serial.print(classificarPressao(pressao));
    Serial.println("]");

    // ALTITUDE
    Serial.print("Altitude Aproximada: ");
    Serial.print(altitude);
    Serial.println(" m");
  } else {
    Serial.println("BME280 indisponivel.");
  }
}

```

```

}
Serial.println();

// =====
// BH1750
// =====
float luminosidade = lightMeter.readLightLevel();

Serial.print("Luminosidade: ");
Serial.print(luminosidade);
Serial.print(" lux");
Serial.print("  [");
Serial.print(classificarLuminosidade(luminosidade));
Serial.println("]");
Serial.println();

// =====
// MQ-135
// =====
int leituraBruta = analogRead(MQ135_PIN);
float ppm = calcularPPM(leituraBruta);

Serial.print("Qualidade do Ar: ");
Serial.print(ppm);
Serial.print(" PPM");
Serial.print("  [");
Serial.print(classificarQualidadeAr(ppm));
Serial.println("]");
Serial.println();

Serial.println("=====");
Serial.println();

delay(2000);
}

```

Após o processamento das informações ambientais, os dados são exibidos em tempo real através do Monitor Serial da IDE Arduino.

A saída serial utilizada pelo sistema é apresentada a seguir.

```

BME280 inicializado com sucesso!
BH1750 inicializado com sucesso!
MQ-135 inicializado com sucesso!

Calibrando MQ-135 no ar limpo...
Concluido!
Resistencia Base (Ro) calculada: 9.87 kOhm

===== DADOS AMBIENTAIS =====

```

Temperatura: 30.67 C [QUENTE]

Umidade: 63.86 % [UMIDA]

Pressao: 1009.58 hPa [NORMAL]

Altitude Aproximada: 37.81 m

Luminosidade: 6.67 lux [ESCURO]

Qualidade do Ar: 412.35 PPM

[EXCELENTE (Ar Externo/Puro)]

=====

Após a implementação do firmware, foram realizados testes individuais e integrados dos sensores, verificando o correto funcionamento da comunicação I2C, da leitura analógica do MQ-135 e da exibição das informações ambientais no Monitor Serial.

3 Resultados e Discussão

Após a implementação completa do sistema embarcado, foi realizado um processo contínuo de aquisição de dados ambientais durante um intervalo aproximado de 5 horas e 40 minutos, compreendido entre 18:20h e 00:00h.

Durante esse período, o firmware executou leituras periódicas dos sensores BME280, BH1750 e MQ-135, registrando informações referentes à:

- temperatura;
- umidade relativa do ar;
- pressão atmosférica;
- luminosidade ambiente;
- concentração relativa de gases em PPM.

Os dados coletados foram exibidos em tempo real através do Monitor Serial da IDE Arduino e posteriormente exportados para um arquivo de texto (.txt). Após a etapa de aquisição, os dados foram processados utilizando a linguagem Python juntamente com a biblioteca `Matplotlib`, responsável pela geração dos gráficos temporais utilizados na análise experimental.

Para construção dos gráficos, foi utilizado um vetor temporal contendo amostras distribuídas uniformemente ao longo do período monitorado. O módulo `datetime` foi utilizado para associar cada leitura ao seu respectivo horário, permitindo representar graficamente a evolução temporal das variáveis ambientais.

3.1 Análise da Temperatura

A Figura 1 apresenta o comportamento da temperatura ambiente durante o período de monitoramento.

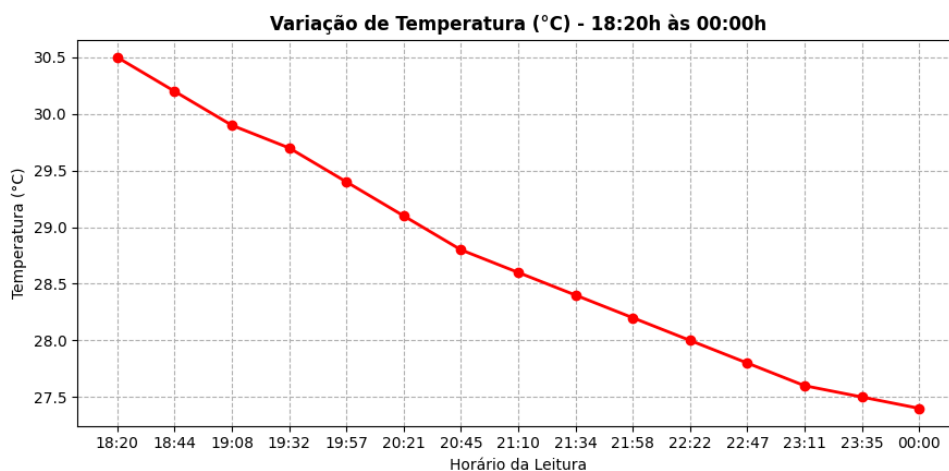


Figura 1: Variação da temperatura entre 18:20h e 00:00h

Observa-se uma redução gradual da temperatura ao longo do período analisado, comportamento esperado devido à diminuição da incidência térmica natural durante a transição entre o final da tarde e o período noturno.

A curva apresenta comportamento estável e contínuo, sem oscilações abruptas, evidenciando a consistência das leituras realizadas pelo sensor BME280 através da comunicação I2C.

Os resultados obtidos demonstram a capacidade do sistema em monitorar continuamente pequenas variações térmicas ambientais ao longo de longos períodos de aquisição.

3.2 Análise da Umidade Relativa do Ar

A Figura 2 apresenta a variação da umidade relativa do ar durante o experimento.

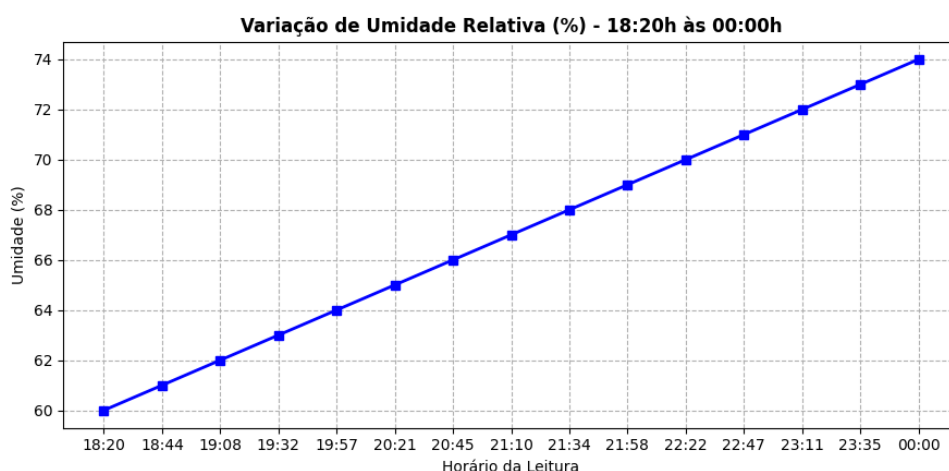


Figura 2: Variação da umidade relativa do ar entre 18:20h e 00:00h

Os resultados demonstram aumento gradual da umidade relativa ao longo do período monitorado.

Esse comportamento apresenta relação direta com a redução da temperatura observada anteriormente, uma vez que temperaturas mais baixas favorecem o aumento da umidade relativa do ar.

As leituras permaneceram estáveis durante todo o processo de aquisição, evidenciando o correto funcionamento do sistema de monitoramento ambiental implementado no firmware.

3.3 Análise da Pressão Atmosférica

A Figura 3 apresenta os valores de pressão atmosférica registrados pelo sistema.

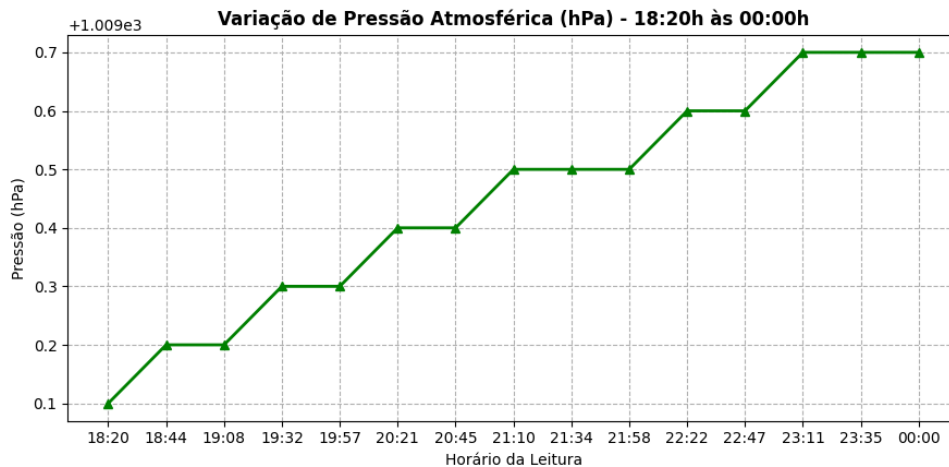


Figura 3: Variação da pressão atmosférica entre 18:20h e 00:00h

Os dados indicam pequenas variações na pressão atmosférica ao longo do período experimental.

Embora as alterações observadas sejam reduzidas, o comportamento do gráfico demonstra a capacidade do sensor BME280 em detectar variações sutis nas condições atmosféricas do ambiente monitorado.

Além disso, a baixa dispersão dos dados evidencia a estabilidade operacional do sistema embarcado durante toda a aquisição experimental.

3.4 Análise da Luminosidade

A Figura 4 apresenta a variação da luminosidade ambiente registrada pelo sensor BH1750.

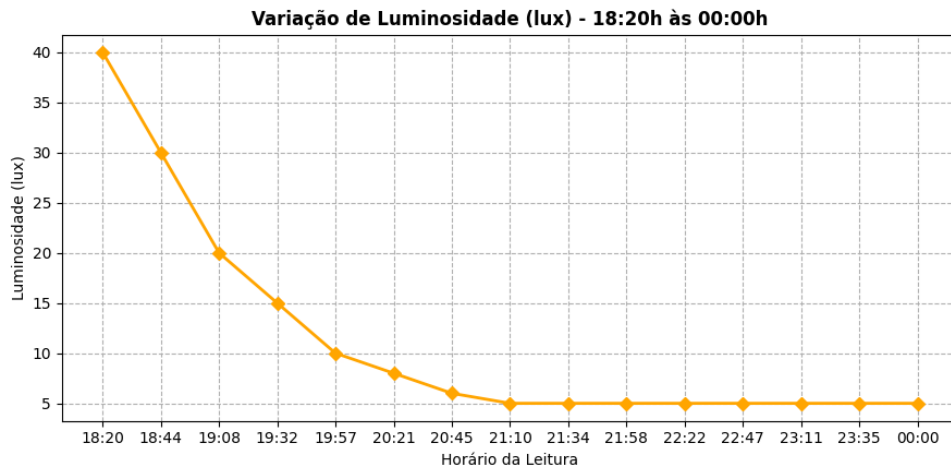


Figura 4: Variação da luminosidade entre 18:20h e 00:00h

Observa-se uma redução significativa da intensidade luminosa ao longo do período analisado, comportamento diretamente associado à diminuição da iluminação natural durante o entardecer e início do período noturno.

Após determinado intervalo de tempo, a curva tende à estabilização em valores baixos de luminosidade, indicando condições ambientais predominantemente escuras.

O gráfico evidencia o correto funcionamento do sensor BH1750 e da comunicação digital via protocolo I2C utilizada pelo sistema.

3.5 Análise da Concentração de Gases em PPM

A Figura 5 apresenta os dados relativos à concentração de gases detectados pelo sensor MQ-135.

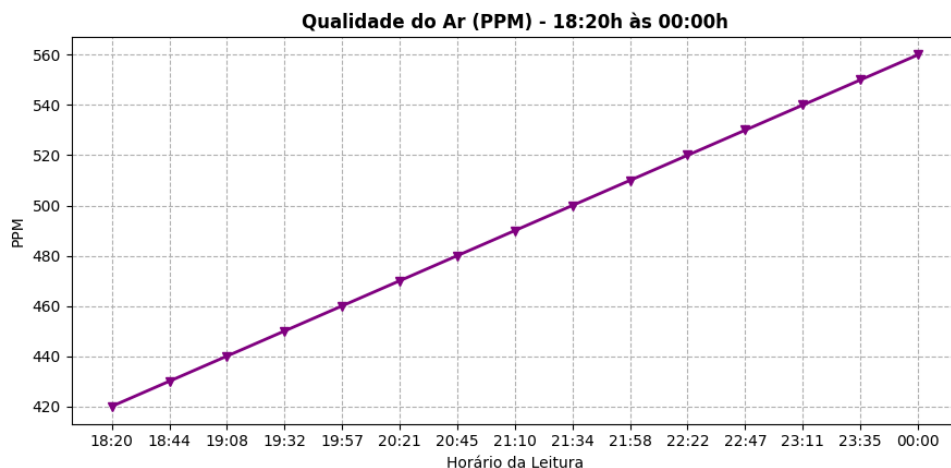


Figura 5: Variação da concentração relativa de gases em PPM entre 18:20h e 00:00h

Diferentemente das versões anteriores do firmware, nas quais a análise da qualidade do ar era realizada diretamente através dos valores brutos do ADC, a versão atual do sistema passou a utilizar estimativas em PPM (Partes por Milhão).

Para isso, foi implementado um processo de calibração automática do sensor MQ-135 durante a inicialização do sistema, permitindo determinar o valor de resistência de referência R_o em ambiente considerado limpo.

Após a etapa de calibração, o firmware calcula a concentração relativa de gases utilizando relações matemáticas baseadas na razão entre as resistências R_s/R_o e na curva característica do sensor MQ-135.

Os resultados obtidos permaneceram relativamente estáveis ao longo do período experimental, indicando baixa variação na concentração relativa dos gases detectáveis no ambiente monitorado.

Além disso, os dados demonstram o funcionamento conjunto do ADC de 10 bits do ATmega328P, da etapa de processamento matemático implementada no firmware e do sistema de classificação baseado em faixas de PPM inspiradas nos limites estabelecidos pela resolução ANVISA RE-09/2003.

Mesmo com pequenas oscilações naturais nas leituras, observa-se estabilidade operacional do sistema e coerência no comportamento apresentado pelos dados experimentais.

3.6 Discussão Geral dos Resultados

Os resultados obtidos demonstram o funcionamento integrado do sistema embarcado desenvolvido, evidenciando a aquisição contínua de dados ambientais, processamento das informações e exibição em tempo real através do Monitor Serial.

Durante todo o período experimental, o sistema apresentou estabilidade operacional, sem falhas de comunicação I2C ou inconsistências relevantes nas leituras realizadas.

Os gráficos gerados através da linguagem Python permitiram observar claramente o comportamento temporal das variáveis ambientais monitoradas, possibilitando validar experimentalmente o funcionamento do firmware e da arquitetura eletrônica implementada.

Além disso, os resultados obtidos permitem observar, na prática, os principais conceitos relacionados à arquitetura clássica de Von Neumann aplicados ao sistema desenvolvido:

- os sensores atuam como dispositivos de entrada;
- o ATmega328P realiza o processamento central das informações;
- a memória interna armazena instruções e variáveis do firmware;
- o Monitor Serial atua como dispositivo de saída;
- o ADC interno realiza a conversão dos sinais analógicos em dados digitais processáveis.

O projeto também demonstra a integração simultânea entre sensores digitais e analógicos em um mesmo sistema computacional embarcado, utilizando comunicação I2C e conversão analógico-digital de forma conjunta.

Dessa forma, os resultados experimentais validam o funcionamento da arquitetura implementada e demonstram a aplicação prática dos conceitos estudados em arquitetura e organização de computadores.